QUIZ PERTEMUAN 6

PEMROGRAMAN BERORIENTASI OBJEK

Oleh:

Zeyan Fahluzi Alhariz

250660221033

SOAL

1. Konsep Abstraction

Jelaskan pengertian abstraction dalam OOP dan berikan satu contoh antarmuka yang menyembunyikan detail implementasi.

Jawaban:

- **Pengertian Abstraction:** Abstraction (abstraksi) adalah prinsip dalam OOP yang bertujuan untuk menyembunyikan detail kerumitan proses internal dari suatu objek atau sistem, dan hanya mengekspos fungsionalitas utama (antarmuka) yang relevan dan penting saja kepada pengguna. Tujuannya adalah untuk menyederhanakan penggunaan suatu sistem.
- **Contoh Antarmuka:** Fitur penyalakan mesin mobil menggunakan tombol *Start/Stop Engine*. Pengemudi (pengguna) hanya perlu menekan tombol tersebut (antarmuka) untuk menyalakan mobil, tanpa perlu mengetahui atau memikirkan bagaimana sistem kelistrikan menyala, bagaimana pompa bahan bakar bekerja, atau bagaimana busi memercikkan api di dalam mesin (detail implementasi).

2. Abstract Class dan Abstract Method

Jelaskan perbedaan Abstract Class dan Abstract Method. Mengapa abstract class tidak dapat diinstalasi langsung?

Jawaban:

- **Perbedaan:**
- **Abstract Class:** Sebuah kelas kerangka/cetak biru yang tidak dapat digunakan untuk membuat objek nyata (tidak bisa diinstansiasi). Kelas ini hanya berfungsi sebagai kelas dasar (*base class*) yang mewariskan sifat dan memaksa aturan pada kelas turunannya.
- **Abstract Method:** Sebuah *method* yang dideklarasikan di dalam *abstract class* namun tidak memiliki isi blok kode (implementasi kosong, biasanya hanya berisi `pass`). *Method* ini harus (wajib) diimplementasikan ulang di kelas turunannya.
- **Mengapa abstract class tidak dapat diinstansiasi langsung?** Karena kelas tersebut belum "lengkap". Adanya *abstract method* berarti ada perilaku yang belum didefinisikan. Jika program mengizinkan pembuatan objek dari *abstract class*, maka akan terjadi kebingungan (atau *error* sistem) apabila *method* yang kosong/belum ada implementasinya tersebut dipanggil oleh pengguna.

3. Prediksi Output

```python
from abc import ABC, abstractmethod

class Pengguna(ABC):
    def __init__(self, nama):
        self.nama = nama

    @abstractmethod
    def tampilkan_peran(self):
        pass

class Mahasiswa(Pengguna):
    def tampilkan_peran(self):
        return "Mahasiswa"

print(Mahasiswa("Budi").tampilkan_peran())
```

Tuliskan output dari program tersebut dan jelaskan peran @abstractmethod.

Jawaban:

Output yang muncul:

Mahasiswa

Peran `@abstractmethod`: Dekorator `@abstractmethod` bertugas sebagai "kontrak" yang memaksa dan mewajibkan setiap kelas turunan (*subclass*) untuk mengimplementasikan (*override*) *method* tersebut. Dalam kode ini, ia memaksa kelas `Mahasiswa` untuk menyediakan blok kode nyata untuk `tampilkan_peran()`. Jika kelas turunan tidak mengimplementasikan *method* yang ditandai dengan `@abstractmethod`, maka kelas turunan tersebut akan otomatis dianggap sebagai kelas abstrak juga, dan Python akan memunculkan *error* (berupa `TypeError`) ketika kelas turunan tersebut mencoba diinstansiasi.

4. Error pada Abstract Class

```python
from abc import ABC, abstractmethod

class Pengguna(ABC):
    @abstractmethod
    def tampilkan_peran(self):
        pass

class Mahasiswa(Pengguna):
    pass

mahasiswa = Mahasiswa()
```

Apa yang terjadi ketika mahasiswa() dijalankan? Jelaskan alasan teknisnya.

Jawaban:

- Apa yang terjadi: Ketika `Mahasiswa()` dijalankan, program akan berhenti dan menghasilkan `TypeError`. Pesan *error* biasanya berbunyi *Can't instantiate abstract class Mahasiswa with abstract method tampilkan_peran*.

- Alasan Teknis: Kelas `Pengguna` adalah sebuah *abstract class* yang mendefinisikan *abstract method* bernama `tampilkan_peran()`. Kelas `Mahasiswa` mewarisi kelas `Pengguna`, tetapi ia hanya berisi `pass` dan tidak mengimplementasikan ulang (*override*) *abstract method* tersebut. Oleh karena itu, Python menganggap `Mahasiswa` juga masih sebagai *abstract class* yang belum lengkap, sehingga instansiasi (pembuatan objek) tidak diizinkan.

5. Merancang Abstract Class

Tuliskan abstract class transaksi dengan abstract method proses() , lalu buat subclass TransaksiTunai yang mengimplementasikan method tersebut. Jelaskan manfaat kontrak perilaku pada rancangan tersebut.

Jawaban:

Kode implementasi:

```python
from abc import ABC, abstractmethod


class Transaksi(ABC):
    @abstractmethod
    def proses(self):
        pass


class TransaksiTunai(Transaksi):
    def proses(self):
        print("Memproses transaksi menggunakan uang tunai.")

```

Manfaat Kontrak Perilaku pada Rancangan Tersebut: Dengan menggunakan *abstract class*, `Transaksi` bertindak sebagai "kontrak" ketat yang memaksa dan mewajibkan setiap kelas turunannya (seperti `TransaksiTunai`, atau nantinya `TransaksiKredit`, `TransaksiTransfer`) untuk wajib memiliki *method* `proses()`. Manfaat utamanya adalah menjaga konsistensi dan keamanan struktur program. Sistem lain yang menggunakan kelas-kelas transaksi ini tidak perlu khawatir terjadi *error* (seperti `AttributeError`) saat memanggil *method* `.proses()` karena ketersediaan *method* tersebut sudah dijamin oleh *abstract class*.